



데이터베이스 설계 문서

1. 테이블 정의

- DB명: `ddoksori_db`
- DBMS: PostgreSQL

1.1. USER (내부 사용자)

본 테이블은 서비스에 가입한 내부 사용자 기본 정보를 관리한다.
모든 사용자 관련 데이터의 기준 엔티티이며,
인증, 세션, 채팅 데이터는 본 테이블을 참조한다.

컬럼 정의

컬럼명(KO)	컬럼명(EN)	Data Type	NULL	Default	역할	제약	비고
사용자 ID	id	TEXT	N	UUID	PK		내부 사용자 고유 식별자
사용자 상태	status	user_status	N	active		ENUM	active / blocked / deleted
사용자 권한	role	TEXT	N	user		ENUM	user / admin
닉네임	nickname	TEXT	Y				사용자 표시명
정규화 닉네임	nickname_norm	TEXT	Y			UNIQUE(부분)	중복 검사용
생성일시	created_at	TIMESTAMP	N	now()			
마지막 로그인	last_login_at	TIMESTAMP	Y				
삭제일시	deleted_at	TIMESTAMP	Y				소프트 삭제

제약조건

- UNIQUE
 - `nickname_norm` UNIQUE WHERE `status = 'active'`

1.2. USER_AUTH_PROVIDER (소셜 로그인 정보)

본 테이블은 사용자의 소셜 로그인 인증 정보를 관리한다.
외부 인증 정보는 내부 사용자 정보와 분리하여 관리되며,
각 사용자는 하나의 소셜 계정만 연결할 수 있다.

컬럼 정의

컬럼명(KO)	컬럼명(EN)	Data Type	NULL	Default	역할	제약	비고
인증 ID	id	TEXT	N	UUID	PK		
사용자 ID	user_id	TEXT	N		FK→user.id	UNIQUE	
소셜 제공자	provider	TEXT	N			ENUM	google / kakao / naver
소셜 사용자 ID	provider_user_id	TEXT	N				소셜 고유 ID
소셜 이메일	provider_email	TEXT	Y				
생성일시	created_at	TIMESTAMP	N	now()			
수정일시	updated_at	TIMESTAMP	N	now()			

제약조건

- UNIQUE
 - `(provider, provider_user_id)`

1.3. USER_SESSION (로그인 세션)

본 테이블은 사용자의 로그인 유지 정보(세션)를 관리한다.
하나의 사용자는 여러 기기 또는 브라우저에서 동시에 로그인할 수 있으며,
세션 단위로 만료 및 폐기 상태를 관리한다.

컬럼 정의

컬럼명(KO)	컬럼명(EN)	Data Type	NULL	Default	역할	제약	비고
세션 ID	id	TEXT	N	UUID	PK		
사용자 ID	user_id	TEXT	N		FK→user.id		

컬럼명(KO)	컬럼명(EN)	Data Type	NULL	Default	역할	제약	비고
세션 타입	session_type	TEXT	N	web		ENUM	web / mobile
리프레시 토큰 해시	refresh_token_hash	TEXT	Y				
생성일시	created_at	TIMESTAMP	N	now()			
만료일시	expired_at	TIMESTAMP	N				
폐기일시	revoked_at	TIMESTAMP	Y				
활성 여부	is_active	BOOLEAN	N	true			

1.4. CHAT (챗봇 대화방)

본 테이블은 사용자와 챗봇 간의 **대화방 단위 정보**를 관리한다.

대화방은 사용자 기준으로 생성되며,

대화 목록 정렬 및 상태 관리를 위한 메타데이터를 저장한다.

컬럼 정의

컬럼명(KO)	컬럼명(EN)	Data Type	NULL	Default	역할	제약	비고
채팅방 ID	id	TEXT	N	UUID	PK		
사용자 ID	user_id	TEXT	N		FK→user.id		
제목	title	TEXT	Y				
상태	status	chat_status	N	active		ENUM	active / archived / deleted
생성일시	created_at	TIMESTAMP	N	now()			
마지막 메시지 시각	last_message_at	TIMESTAMP	Y				
LangSmith Thread ID	langsmith_thread_id	TEXT	Y				

1.5. CHAT_MESSAGE (채팅 메시지)

본 테이블은 챗봇 대화방 내에서 발생하는 **모든 메시지**를 관리한다.

메시지는 대화방 내부에서만 의미를 가지며,

저장 순서를 기준으로 대화 히스토리를 구성한다.

컬럼 정의

컬럼명(KO)	컬럼명(EN)	Data Type	NULL	Default	역할	제약	비고
메시지 ID	id	TEXT	N	UUID	PK		
채팅방 ID	chat_id	TEXT	N		FK→chat.id		
역할	role	chat_message_status	N			ENUM	user / assistant / system / tool
순서	sequence	INT	N			UNIQUE	
내용	content	TEXT	N				
생성일시	created_at	TIMESTAMP	N	now()			
에이전트 ID	agent_id	TEXT	Y				

제약조건

- **UNIQUE**
 - (chat_id, sequence)

1.6. COMMUNITY_CATEGORY (커뮤니티 카테고리)

본 테이블은 커뮤니티 게시글을 분류하기 위한 **카테고리 정보**를 관리한다.

카테고리명과 정렬 순서를 저장하며, 게시글은 카테고리를 참조한다.

컬럼 정의

컬럼명(KO)	컬럼명(EN)	Data Type	NULL 여부	Default	역할	제약	비고
카테고리 ID	id	TEXT	N	UUID	PK		
카테고리명	category_name	TEXT	N			UNIQUE	
정렬 순서	sort_order	INT	Y			UNIQUE	목록 정렬용
생성일시	created_at	TIMESTAMP	N	now()			

1.7. COMMUNITY_POST (커뮤니티 게시글)

본 테이블은 커뮤니티의 **게시글 정보**를 관리한다.

게시글은 작성자(**user**)와 카테고리(**community_category**)에 귀속되며,

공개/숨김/삭제 상태를 `status` 로 관리한다(소프트 삭제 정책).

컬럼 정의

컬럼명(KO)	컬럼명(EN)	Data Type	NULL	Default	역할	제약	비고
게시글 ID	id	TEXT	N	UUID	PK		
작성자 ID	user_id	TEXT	N		FK→user.id		
카테고리 ID	category_id	TEXT	N		FK→community_category.id		
제목	title	TEXT	N				
내용	content	TEXT	N				
조회수	view_count	INT	N	0			
상태	status	community_content_status	N	normal		ENUM	normal/hidden/delet
생성일시	created_at	TIMESTAMP	N	now()			
수정일시	updated_at	TIMESTAMP	N	now()			시스템 갱신
편집일시	edited_at	TIMESTAMP	Y				사용자 편집
삭제일시	deleted_at	TIMESTAMP	Y				Soft Delete

인덱스

```
-- 카테고리별 최신 게시글 조회
CREATE INDEX IDX_POST_CATEGORY_CREATED_AT
  ON COMMUNITY_POST (category_id, created_at DESC);

-- 특정 사용자 작성글 최신순 조회
CREATE INDEX IDX_POST_USER_CREATED_AT
  ON COMMUNITY_POST (user_id, created_at DESC);

-- 카테고리 + 상태 + 최신순 조회
CREATE INDEX IDX_POST_CATEGORY_STATUS_CREATED_AT
  ON COMMUNITY_POST (category_id, status, created_at DESC);
```

상태/삭제 일관성

- CHECK: `(status = 'deleted') = (deleted_at IS NOT NULL)`
- `status='deleted'` 변경 시 COMMUNITY_COMMENT 테이블에서 해당 post_id를 외래키로 갖는 모든 댓글에 대해 `status='deleted'`

편집 시간 정책

- `edited_at` 은 사용자가 제목/내용을 변경한 경우에만 갱신
(`updated_at` 은 시스템 갱신, `edited_at` 은 사용자 편집 기록용으로 분리)

탈퇴한 작성자의 게시글 관리 정책

- 사용자가 탈퇴해도(`user.status='deleted'`) 게시글은 삭제하지 않고 보존한다.
- UI에서는 작성자 닉네임을 user 테이블에서 참조 필요 없이 `"탈퇴한 사용자"` 로 표시한다.
- FK(`user_id`)는 유지하여 데이터 정합성을 보장한다.

1.8. COMMUNITY_COMMENT (커뮤니티 댓글)

본 테이블은 게시글에 달리는 댓글 및 대댓글을 관리한다.

대댓글은 `parent_comment_id` 를 통해 자기참조(Self FK) 구조로 구현하며, 대댓글은 1단까지만 허용(대댓글의 대댓글 금지)한다.

삭제는 게시글과 동일하게 소프트 삭제로 관리한다.

컬럼 정의

컬럼명(KO)	컬럼명(EN)	Data Type	NULL	Default	역할	제약	비고
댓글 ID	id	TEXT	N	UUID	PK		
게시글 ID	post_id	TEXT	N		FK→community_post.id		
작성자 ID	user_id	TEXT	N		FK→user.id		
부모 댓글 ID	parent_comment_id	TEXT	Y	NULL	Self FK		대댓글
내용	content	TEXT	N				
상태	status	community_content_status	N	normal		ENUM	normal/hidden/delete
생성일시	created_at	TIMESTAMP	N	now()			
수정일시	updated_at	TIMESTAMP	N	now()			시스템 갱신
편집일시	edited_at	TIMESTAMP	Y				사용자 편집
삭제일시	deleted_at	TIMESTAMP	Y			Soft Delete	Soft Delete

인덱스

```
-- 게시물별 댓글을 최신순으로 조회
CREATE INDEX IDX_COMMENT_POST_CREATED_AT
ON COMMUNITY_COMMENT (post_id, created_at DESC);

-- 게시물 내 댓글을 상태(정상/숨김/삭제)로 필터링하여 최신순 조회
CREATE INDEX IDX_COMMENT_POST_STATUS_CREATED_AT
ON COMMUNITY_COMMENT (post_id, status, created_at DESC);

-- 특정 댓글의 대댓글(부모 댓글 기준) 조회
CREATE INDEX IDX_COMMENT_PARENT_COMMENT_ID
ON COMMUNITY_COMMENT (parent_comment_id);

-- 특정 사용자가 작성한 댓글을 최신순으로 조회
CREATE INDEX IDX_COMMENT_USER_CREATED_AT
ON COMMUNITY_COMMENT (user_id, created_at DESC);
```

상태/삭제 일관성

- CHECK: `(status = 'deleted') = (deleted_at IS NOT NULL)`

Self FK 기본 제약

- CHECK: `parent_comment_id IS NULL OR parent_comment_id <> id` (자기 참조 금지)

대댓글 1단 제한

- 정책: “대댓글의 대댓글 작성 금지”
- 구현: 대댓글의 `parent_comment_id` 가 가리키는 부모 댓글의 `parent_comment_id IS NULL` 강제

동일 게시물 내 대댓글 정합성

- 부모 댓글의 `post_id` 와 일치

탈퇴한 작성자의 댓글 관리 정책

- 댓글은 삭제하지 않고 보존한다.
- UI에서는 작성자 닉네임을 user 테이블에서 참조 필요 없이 “탈퇴한 사용자” 로 표시한다.
- 대댓글 구조/이력은 그대로 보존한다.

1.9. COMMUNITY_POST_LIKE (게시글 추천)

본 테이블은 사용자와 게시물 간 **추천(좋아요)** 관계를 관리한다.

추천은 사용자-게시글의 다대다 관계이므로 조인 테이블로 분리한다.

컬럼 정의

컬럼명(KO)	컬럼명(EN)	Data Type	NULL 여부	Default 값	역할	제약	비고
추천 ID	id	TEXT	N	UUID	PK		
게시글 ID	post_id	TEXT	N		FK→community_post.id		
사용자 ID	user_id	TEXT	N		<u>FK→user.id</u>		
생성일시	created_at	TIMESTAMP	N	now()			

제약조건

- UNIQUE** `(post_id, user_id)` → 동일 게시물 중복 추천 방지

인덱스

```
-- 사용자가 추천(좋아요)한 게시글을 최신순으로 조회하기 위한 인덱스
CREATE INDEX IDX_COMMUNITY_POST_LIKE_USER_CREATED_AT
ON COMMUNITY_POST_LIKE (user_id, created_at DESC);
```

탈퇴한 사용자 추천 관리 정책

- 추천 데이터는 물리 삭제하지 않고 유지
- 집계에도 포함

1.10. COMMUNITY_REPORT (커뮤니티 신고)

본 테이블은 커뮤니티에서 발생하는 **신고 이벤트**를 관리한다.

신고 대상은 게시물/댓글로 다양하므로 `target_type + target_id` 조합으로 식별하며,

즉, “신고 1회 = row 1개”이며, 다수 사용자가 동일 게시물/댓글을 신고하면 해당 타겟에 대해 여러 row가 생성된다.

운영 처리 상태(**status**)와 처리 이력(처리자/처리 시각/메모)을 저장한다.

컬럼 정의

컬럼명(KO)	컬럼명(EN)	Data Type	NULL	Default	역할	제약	비고
신고 ID	id	TEXT	N	UUID	PK		
신고자 ID	reporter_user_id	TEXT	N		FK→user.id		
대상 유형	target_type	community_report_target_type	N			ENUM	post/comment
대상 ID	target_id	TEXT	N		Polymorphic Key		
신고 사유	reason_code	TEXT	N				
상세 사유	reason_text	TEXT	Y				
처리 상태	status	community_report_status	N	pending		ENUM	pending/ resolved/ rejected
생성일시	created_at	TIMESTAMP	N	now()			
처리일시	handled_at	TIMESTAMP	Y				
처리자 ID	handled_by_user_id	TEXT	Y		FK→user.id		관리자
처리 메모	resolution_note	TEXT	Y				

제약조건

- **UNIQUE** (**reporter_user_id, target_type, target_id**) → **중복 신고 방지**. 동일 신고자가 동일 타겟에 대해 중복 신고 불가(“1인 1타겟 1회 신고”)
- **CHECK**
 - 처리 상태에 대해
 - **status = 'pending'** 인 경우
 - **handled_at IS NULL**
 - **handled_by_user_id IS NULL**
 - **status IN ('resolved','rejected')** 인 경우
 - **handled_at IS NOT NULL**
 - **handled_by_user_id IS NOT NULL**
- **타겟 무결성 보장**: 신고 row 생성 시 대상 존재 여부 확인(post/comment 테이블 조회)

인덱스

```
-- 신고 상태별 목록을 최신순으로 조회 (운영자 모니터링용)
CREATE INDEX IDX_COMMUNITY_REPORT_STATUS_CREATED_AT
ON COMMUNITY_REPORT (status, created_at DESC);

-- 특정 게시물/댓글(target)에 대한 신고 내역을 모아서 조회
CREATE INDEX IDX_COMMUNITY_REPORT_TARGET_TYPE_ID
ON COMMUNITY_REPORT (target_type, target_id);

-- 특정 사용자의 신고 이력을 최신순으로 조회 (악성 사용자 추적용)
CREATE INDEX IDX_COMMUNITY_REPORT_REPORTER_USER_CREATED_AT
ON COMMUNITY_REPORT (reporter_user_id, created_at DESC);
```

케이스(타겟별) 운영 조회 방식

케이스 테이블을 두지 않으므로, 운영 화면의 “신고된 게시물/댓글 목록(타겟별 1줄)”은 아래 방식으로 구성한다.

- **GROUP BY (target_type, target_id)** 로
 - 신고 건수 **COUNT(*)**
 - 최근 신고 시각 **MAX(created_at)**
 - 미처리 건수 **COUNT(*) FILTER (WHERE status='pending')**
- 등을 집계

또한 타겟에 대한 조치(숨김/삭제/유지) 후에는 운영 편의를 위해

- 해당 타겟의 **pending** 신고 row를 일괄 **resolved/rejected** 로 업데이트(정책에 맞게)하는 운영을 권장한다.
(단일 테이블에서 “케이스 종결”을 표현하는 현실적인 방법)

탈퇴한 사용자 신고 건 관리 정책

- 신고 데이터는 물리 삭제하지 않고 보존
- 신고자/처리자가 탈퇴해도 기록 유지
- UI에서는 사용자 표시를 “**탈퇴한 사용자**” 로 비식별 처리

1.11. ADMIN_ACTION_LOG (관리자 행위 로그)

본 테이블은 커뮤니티 운영 과정에서 발생하는 **관리자(ADMIN) 행위 이력**을 기록한다.

관리자 권한(버튼 노출/접근 제어)은 애플리케이션 코드에서 수행하되, 실제 실행된 운영 행위를 본 테이블에 저장하여 **감사(Audit)·추적성·운영 안정성**을 확보한다.

예시 행위:

- 게시글/댓글 삭제(소프트 삭제), 숨김 처리
- 신고 처리 상태 변경(pending → resolved/rejected)
- 카테고리 생성/수정/정렬 변경(항후)

본 테이블은 “권한”을 저장하지 않고, **행위 이력(누가/무엇을/어디에/왜/언제)**만 저장한다.

컬럼 정의

컬럼명(KO)	컬럼명(EN)	Data Type	NULL	Default	역할	제약	비고
로그 ID	id	TEXT	N	UUID	PK		
행위자 ID	actor_user_id	TEXT	N		FK→user.id		관리자
행위 유형	action_type	admin_action_type	N			ENUM	
대상 유형	target_type	admin_action_target_type	N			ENUM	post/comment/report/category
대상 ID	target_id	TEXT	Y		Polymorphic Key		
사유	reason	TEXT	Y				
추가 정보	payload	JSONB	Y				
생성일시	created_at	TIMESTAMP	N	now()			

인덱스

```
-- 특정 관리자의 행위 이력을 최신순으로 조회
CREATE INDEX IDX_ADMIN_ACTION_LOG_ACTOR_CREATED_AT
ON ADMIN_ACTION_LOG (actor_user_id, created_at DESC);

-- 특정 게시글/댓글/신고(target)에 대한 조치 이력을 시간순으로 추적용
CREATE INDEX IDX_ADMIN_ACTION_LOG_TARGET_TYPE_ID_CREATED_AT
ON ADMIN_ACTION_LOG (target_type, target_id, created_at DESC);

-- 행위 유형별(action_type) 통계 및 감사 로그를 최신순으로 조회
CREATE INDEX IDX_ADMIN_ACTION_LOG_ACTION_TYPE_CREATED_AT
ON ADMIN_ACTION_LOG (action_type, created_at DESC);
```

탈퇴한 관리자 관리 정책

- 일반 사용자가 아닌 “관리자 역할 사용자”도 **user** 레코드로 관리하며, 물리 삭제하지 않는다.
- 탈퇴(비활성) 처리 이후에도 운영 행위 이력은 보존한다.
- UI에서는 actor 표시가 필요하면 **“탈퇴한 관리자”** 로 비식별 표시한다.

ENUM: admin_action_type

```
CREATE TYPE admin_action_type AS ENUM (
    'post_edit', 'post_hide', 'post_delete',
    'comment_edit', 'comment_hide', 'comment_delete',
    'report_resolve', 'report_reject',
    'category_create', 'category_update', 'category_reorder'
);
```

1.12. LAWS (법령 목록)

본 테이블은 국가 법령 API로부터 수집한 **법령의 메타데이터(기본 정보)**를 저장한다.

법령의 고유 식별자, 법령명, 소관 부처, 공포·시행일자 및 개정 유형 등의 정보를 관리하며,

실제 조문 단위 정보는 **law_units** 테이블에서 별도로 관리한다.

컬럼명(KO)	컬럼명(EN)	Data Type	NULL	Default	역할	제약	비고
법령 ID	law_id	TEXT	N		PK		법령에 부여된 고유 식별자
법령명	law_name	TEXT	N				법령의 공식 한글 명칭
법령 구분	law_type	TEXT	Y			ENUM	법률 / 시행령 / 시행규칙
소관 부처	ministry	TEXT	Y				법령을 관할하는 행정기관
공포일자	promulgation_date	DATE	Y				법령 공포 일자
시행일자	enforcement_date	DATE	Y				법령 시행 일자
개정 유형	revision_type	TEXT	Y				일부개정 / 전부개정 / 타법개정 등

컬럼명(KO)	컬럼명(EN)	Data Type	NULL	Default	역할	제약	비고
도메인	domain	TEXT	Y	statute			법령 분류용 도메인

인덱스

```
-- 법령명 기반 탐색 / 자동완성 / 키워드 매핑용
create index IDX_LAWS_LAW_NAME
on LAWS (law_name);

-- 법률 / 시행령 구분 필터링 조회
create index IDX_LAWS_LAW_TYPE
on LAWS (law_type);
```

1.13. LAW_UNITS (법령 조문 단위)

본 테이블은 법령을 구성하는 **조·항·호·목** 단위의 **조문 정보**를 저장한다.

각 조문은 계층 구조를 가지며(`parent_id`),

검색·청킹 대상 여부(`is_indexable`) 및 내부·외부 참조 정보(refs)를 함께 관리한다.

본 테이블은 RAG 검색, 법령 간 참조 분석, 임베딩 청킹의 **핵심 기준 테이블**이다.

컬럼명(KO)	컬럼명(EN)	Data Type	NULL	Default	역할	제약	비고
조문 ID	doc_id	TEXT	N		PK		법령+구조 기반 조문 고유 식별자
법령 ID	law_id	TEXT	N		FK->laws.law_id		
상위 조문 ID	parent_id	TEXT	Y		Self FK		
조문 레벨	level	TEXT	N				article / paragraph / item / subitem 등
인덱싱 여부	is_indexable	BOOLEAN	N				청킹 대상 여부
조 번호	article_no	TEXT	Y				예: 제5조
조 제목	article_title	TEXT	Y				조문 제목
항 번호	paragraph_no	TEXT	Y				예: 제2항
호 번호	item_no	TEXT	Y				예: 제1호
목 번호	subitem_no	TEXT	Y				예: 가목
전체 경로	path	TEXT	Y				법령명 포함 전체 조문 경로
조문 본문	text	TEXT	N				실제 법령 조문 내용
개정 부칙	amendment_note	TEXT	Y				본문 마지막에 붙은 개정 관련 부가 설명
내부 참조	ref_citations_internal	JSONB	N				동일 법령 내 조문 참조
외부 참조	ref_citations_external	JSONB	N				타 법령 조문 참조
언급 법령	mentioned_laws	JSONB	N				본문에 언급된 법령 목록

인덱스

```
-- 특정 법령 내 조문 레벨별 탐색
create index IDX_LAW_UNITS_LAW_ID_LEVEL
on LAW_UNITS (law_id, level);

-- RAG 임베딩 대상 필터링
create index IDX_LAW_UNITS_LAW_ID_IS_INDEXABLE
on LAW_UNITS (law_id, is_indexable);

-- 계층 구조 탐색 (하위 조문 조회)
create index IDX_LAW_UNITS_PARENT_ID
on LAW_UNITS (parent_id);
```

1.14. CRITERIA (분쟁기준 데이터 원천)

본 테이블은 소비자분쟁해결기준 및 관련 지침 데이터를 **원천(source)** 단위로 관리한다.

별표(1~4) 및 지침(전자거래 / 콘텐츠 등)과 같이

서로 성격이 다른 기준 문서들을 하나의 기준 체계로 묶기 위한 상위 테이블이다.

각 원천은 이후 `criteria_units` 테이블에 저장되는 단위 레코드들의 **부모 엔티티** 역할을 한다.

컬럼명(KO)	컬럼명(EN)	Data Type	NULL	Default	역할	제약	비고
원천 ID	source_id	TEXT	N		PK		분쟁기준 원천 고유 식별자 예: <code>table1</code> , <code>table2</code> , <code>table3</code> , <code>table4</code> , <code>ecommerce_guideline</code> , <code>guideline_mcst</code>

컬럼명(KO)	컬럼명(EN)	Data Type	NULL	Default	역할	제약	비고
원천 라벨	source_label	TEXT	Y				화면/문서 표시용 이름 예: 별표1 품목 분류, 전자상거래 소비자보호 지침
설명	description	TEXT	Y				원천 데이터에 대한 간단한 설명
생성 시각	created_at	TIMESTAMPTZ	N	now()			row 생성 시각
수정 시각	updated_at	TIMESTAMPTZ	N	now()			row 수정 시각(업데이트 시 갱신)

1.15. CRITERIA_UNITS (분쟁기준 단위 레코드)

본 테이블은 소비자분쟁해결기준(별표 1~4) 및 관련 지침(전자거래 / 콘텐츠)을

검색·조화·RAG 활용을 위한 단위(unit) 레코드로 저장한다.

업로드된 JSONL 원문 데이터는 `doc` 컬럼(JSONB)에 원형 그대로 보존하며,

검색 및 응답 생성에 직접 활용할 대표 텍스트는 `unit_text` 컬럼으로 별도 관리한다.

각 단위 레코드는 반드시 하나의 분쟁기준 원천(`criteria.source_id`)에 속하며,

원천 단위(`criteria`)와의 관계를 통해 기존 데이터의 출처를 명확히 구분한다.

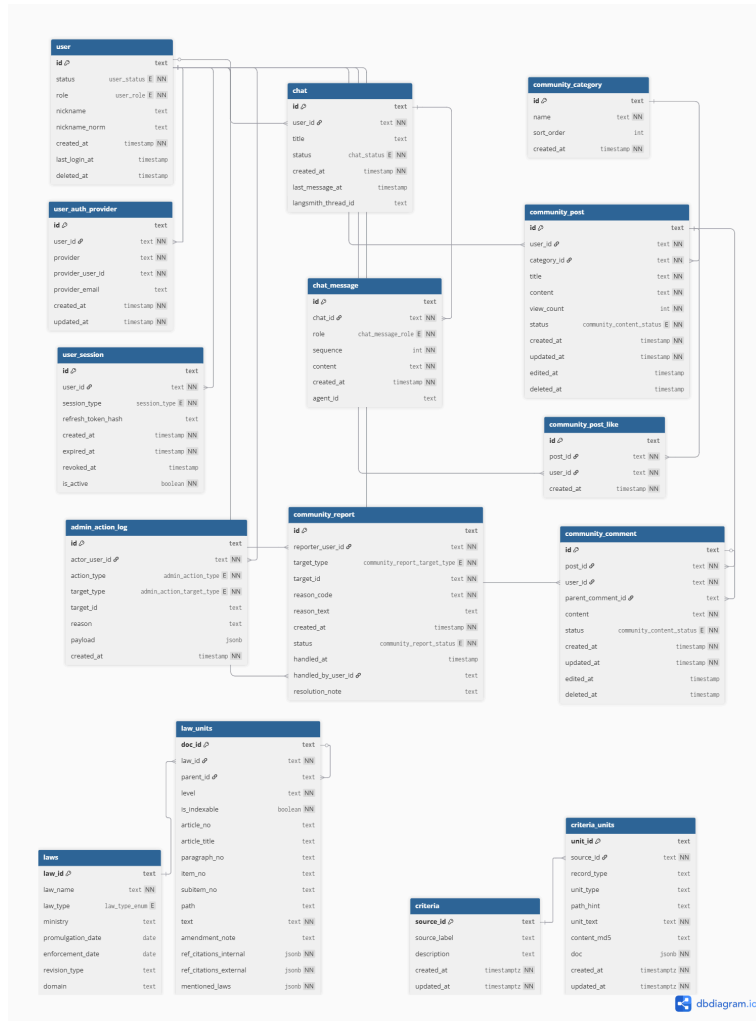
컬럼명(KO)	컬럼명(EN)	Data Type	NULL	Default	역할	제약	비고
단위 ID	unit_id	TEXT	N		PK		분쟁기준 단위 고유 식별자 원천 ID + 내부 식별자 조합
원천 ID	source_id	TEXT	N		FK→criteria.source_id		
레코드 유형	record_type	TEXT	Y				원천별 레코드 구분값 예: <code>item_chunk</code> , <code>rule_chunk</code>
단위 유형	unit_type	TEXT	Y				칭킹 과정에서 생성된 세부 유형 없을 경우 NULL
경로 힌트	path_hint	TEXT	Y				문서/헤딩 기반 경로 요약 검색 결과 설명용
대표 텍스트	unit_text	TEXT	N				검색·RAG 응답에 사용할 대표 텍스트
콘텐츠 해시	content_md5	TEXT	Y			UNIQUE	대표 텍스트 기반 중복 방지용 해시
원문 JSON	doc	JSONB	N				업로드된 JSONL 원형 데이터 전체
생성 시각	created_at	TIMESTAMPTZ	N	now()			row 생성 시각
수정 시각	updated_at	TIMESTAMPTZ	N	now()			row 수정 시각(업데이트 시 갱신)

인덱스

```
-- 원천별 최신 단위 레코드 조회
create index IDX_CRITERIA_UNITS_SOURCE_ID_CREATED_AT
on CRITERIA_UNITS (source_id, created_at);

-- 원천별 레코드 유형 필터링
create index IDX_CRITERIA_UNITS_SOURCE_ID_RECORD_TYPE
on CRITERIA_UNITS (source_id, record_type);
```

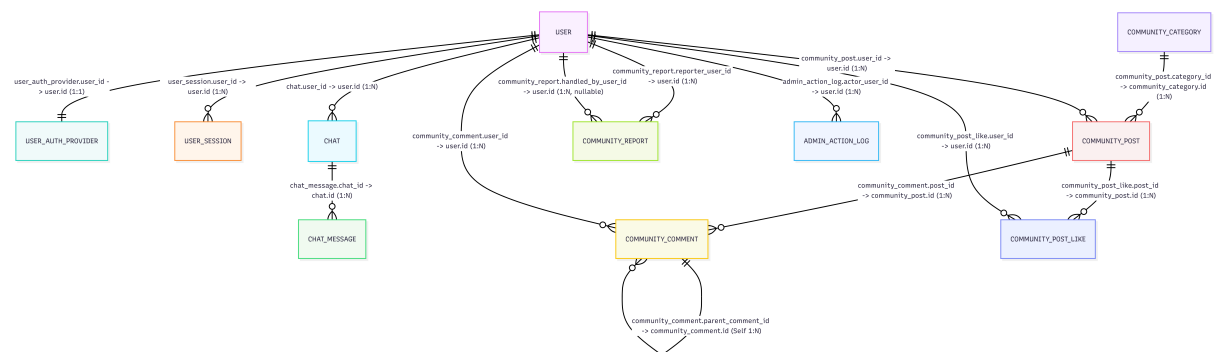
2. ERD



관계 도출 근거

본 데이터 모델의 Entity 간 관계는 서비스의 실제 사용 흐름과

데이터 책임 분리 및 라이프사이클 차이를 기준으로 도출되었다.



1.1 사용자 및 인증 영역 관계 도출

(1) user ↔ user_auth_provider (1:1)

- 본 서비스는 소셜 로그인만 허용하며, 사용자 1명당 하나의 소셜 계정만 사용한다.
- 소셜 제공자별 사용자 식별은 `provider_user_id` 로 수행되지만, 서비스 내부 데이터는 외부 계정 변경과 무관한 **내부 식별자(`user.id`)**를 기준으로 관리한다.
- 이에 따라 인증 정보는 `user_auth_provider` 로 분리하고,

`user_id` 를 기준으로 **1:1** 관계를 형성하였다.

(2) `user` ↔ `user_session` (1:N)

- 한 사용자는 여러 기기 또는 브라우저에서 동시에 로그인할 수 있다.
- 로그인 유지 정보는 사용자 단위로 다수 생성될 수 있으므로, 세션 정보를 별도 엔티티(`user_session`)로 분리하고 `user` 와 **1:N** 관계를 구성하였다.

1.2 챗봇 영역 관계 도출

(3) `user` ↔ `chat` (1:N)

- 사용자는 챗봇과 여러 개의 대화방을 생성할 수 있다.
- 각 대화방은 특정 사용자에게 귀속되며, 사용자 기준으로 대화 이력을 관리해야 하므로 `chat` 은 `user` 와 **1:N** 관계를 갖는다.

(4) `chat` ↔ `chat_message` (1:N)

- 하나의 대화방에는 여러 개의 메시지(질문, 응답, 시스템 메시지, 도구 실행 결과)가 순차적으로 저장된다.
- 메시지는 특정 대화방 내부에서만 의미를 가지며, 단독으로는 독립적인 도메인 역할을 수행하지 않는다.
- 이에 따라 `chat_message` 는 `chat` 과 **1:N** 관계로 도출하였다.

1.3 커뮤니티 영역 관계 도출

(6) `community_category` ↔ `community_post` (1:N)

- 게시글은 카테고리 단위로 분류된다.
- 하나의 카테고리는 여러 개의 게시글을 가질 수 있으므로 **1:N** 관계를 형성한다.
- 게시글은 반드시 하나의 카테고리에 속한다.

관계 컬럼(FK)

- `community_post.category_id → community_category.id`

(7) `user` ↔ `community_post` (1:N)

- 한 사용자는 여러 개의 게시글을 작성할 수 있다.
- 게시글은 반드시 작성자에게 귀속되므로 `user` 와 `community_post` 는 **1:N** 관계이다.
- 사용자가 탈퇴하더라도 게시글은 물리 삭제하지 않고 보존한다.

관계 컬럼(FK)

- `community_post.user_id → user.id`

(8) `community_post` ↔ `community_comment` (1:N)

- 하나의 게시글에는 여러 개의 댓글이 작성될 수 있다.
- 댓글은 게시글 단위로 관리되므로 **1:N** 관계를 도출하였다.

관계 컬럼(FK)

- `community_comment.post_id → community_post.id`

(9) `user` ↔ `community_comment` (1:N)

- 한 사용자는 여러 개의 댓글을 작성할 수 있다.
- 댓글은 작성자에게 귀속되므로 `user` 와 `community_comment` 는 **1:N** 관계이다.
- 작성자가 탈퇴하더라도 댓글은 물리 삭제하지 않고 보존한다.

관계 컬럼(FK)

- `community_comment.user_id → user.id`

(10) `community_comment` ↔ `community_comment` (Self 1:N)

- 댓글은 대댓글 구조를 가질 수 있다.
- 이를 위해 `parent_comment_id` 를 사용하여 자기 참조(Self FK) 관계를 형성하였다.
- 대댓글은 1단까지만 허용한다(대댓글의 대댓글 금지).

관계 컬럼(FK)

- `community_comment.parent_comment_id → community_comment.id` (nullable)

(11) `community_post` ↔ `community_post_like` (1:N)

- 게시글 추천(좋아요)은 사용자와 게시글 간 **M:N 관계**이므로 조인 테이블로 분리하였다.
- 하나의 게시글은 여러 추천을 가질 수 있다.

관계 컬럼(FK)

- `community_post_like.post_id` → `community_post.id`

(12) `user` ↔ `community_post_like` (1:N)

- 한 사용자는 여러 게시글을 추천할 수 있다.
- 동일 사용자가 동일 게시글을 중복 추천하지 못하도록 (`post_id, user_id`)에 UNIQUE 제약을 둔다.

관계 컬럼(FK)

- `community_post_like.user_id` → `user.id`

(13) `user` ↔ `community_report` (1:N, 신고자)

- 한 사용자는 여러 건의 신고를 생성할 수 있다.
- 동일 신고자가 동일 타겟에 대해 중복 신고하지 못하도록 제약을 둔다.

관계 컬럼(FK)

- `community_report.reporter_user_id` → `user.id`

(14) `user` ↔ `community_report` (1:N, 처리자)

- 관리자는 여러 신고 건을 처리할 수 있다.
- 처리자 정보는 신고가 처리된 경우에만 기록된다.

관계 컬럼(FK)

- `community_report.handled_by_user_id` → `user.id` (nullable)

(15) `community_report` ↔ 신고 대상 엔티티 (N:1, 다형 관계)

- 신고 대상은 게시글 또는 댓글이 될 수 있다.
- 이를 위해 단일 FK 대신 `target_type + target_id` 조합을 사용하였다.
- 본 관계는 **물리적 FK로 강제하지 않고 논리적 관계로 관리한다.**

논리적 관계 규칙

- `target_type = 'post'`
→ `target_id` 는 `community_post.id` 를 참조
- `target_type = 'comment'`
→ `target_id` 는 `community_comment.id` 를 참조

(16) `user` ↔ `admin_action_log` (1:N)

- 관리자는 여러 운영 행위를 수행할 수 있다.
- 본 테이블은 권한이 아닌 **실제 실행된 운영 행위 이력**만을 기록한다.

관계 컬럼(FK)

- `admin_action_log.actor_user_id` → `user.id`

1.4 법령 영역 관계 도출

(17) `laws` ↔ `law_units` (1:N)

- 하나의 법령(`laws`)은 여러 조문 단위(`law_units`: 조/항/호/목)를 포함한다.
- 각 조문 단위는 반드시 특정 법령에 속하므로 `law_units.law_id` 가 `laws.law_id` 를 참조하는 **1:N 관계**를 형성하였다.

(18) `law_units` ↔ `law_units` (Self 1:N)

- 조문은 계층 구조(조 → 항 → 호 → 목)로 구성되며, 하위 단위는 상위 단위에 종속된다.
- 이를 표현하기 위해 `law_units.parent_id` 가 `law_units.doc_id` 를 참조하도록 설계하여, 상위 조문 1개에 하위 조문 N개가 연결되는 자기참조 관계(Self FK)를 형성하였다.

1.5 분쟁기준 영역 관계 도출

(19) `criteria` ↔ `criteria_units` (1:N)

- 하나의 분쟁기준 원천(`criteria`)은 여러 개의 분쟁기준 단위 레코드(`criteria_units`)를 포함한다.
- 분쟁기준 원천은 별표(1~4) 및 지침(전자거래 / 콘텐츠 등)과 같이 **문서 단위 기준 집합**을 의미하며, 각 문서는 다수의 검색·조회 단위(unit)로 분해되어 관리된다.

- 각 단위 레코드는 반드시 하나의 원천에 속하므로 `criteria_units.source_id` 가 `criteria.source_id` 를 참조하는 **1:N 관계**를 형성하였다.

Entity 간 관계 및 연결 조건

2.1 사용자 및 인증

- `user.id = user_auth_provider.user_id`
→ 사용자 1명당 소셜 인증 정보 1개 (1:1)
- `(provider, provider_user_id)` UNIQUE
→ 동일 소셜 계정 중복 가입 방지
- `user.id = user_session.user_id`
→ 사용자 다중 로그인(여러 기기/브라우저) 허용 (1:N)
- `user.role (user / admin)`
→ 관리자 권한 구분 (운영 기능 제어)
- `user.nickname_norm` (부분 유니크 인덱스)
→ `status='active'` 사용자에게 한해 닉네임 중복 방지

2.2 챗봇

- `user.id = chat.user_id`
→ 사용자별 대화방 관리 (1:N)
- `chat.id = chat_message.chat_id`
→ 대화방 내부 메시지 관리 (1:N)
- `(chat_id, sequence)` UNIQUE
→ 동일 대화방 내 메시지 순서 중복 방지 (비동기/스트리밍 대비)
- `chat.langsmith_thread_id`
→ 대화방 단위 LangSmith Tracing 연결
- `chat_message.agent_id`
→ 메시지 생성 주체(에이전트/툴) 추적

2.3 커뮤니티 (Entity 간 관계 및 연결 조건)

카테고리 ↔ 게시물

- `community_category.id = community_post.category_id`
→ 게시물은 반드시 하나의 카테고리에 속한다.

사용자 ↔ 게시물

- `user.id = community_post.user_id`
→ 게시물은 반드시 작성자(`user`)에게 귀속된다.

게시물 ↔ 댓글

- `community_post.id = community_comment.post_id`
→ 댓글은 게시물 단위로 관리된다.

사용자 ↔ 댓글

- `user.id = community_comment.user_id`
→ 댓글은 반드시 작성자(`user`)에게 귀속된다.

댓글 ↔ 대댓글 (Self FK)

- `community_comment.id = community_comment.parent_comment_id` (nullable)
→ 대댓글인 경우에만 `parent_comment_id` 가 설정된다.
→ 자기 자신을 부모로 가질 수 없으며, 대댓글은 1단까지만 허용한다.

게시물 ↔ 좋아요 (`community_post_like`)

- `community_post.id = community_post_like.post_id`
- `user.id = community_post_like.user_id`
- **UNIQUE 제약**
 - `(post_id, user_id)`

→ 동일 사용자의 동일 게시글 중복 추천 방지

사용자 ↔ 신고 (`community_report`)

- 신고 생성자:
 - `user.id = community_report.reporter_user_id`
- 신고 처리자(관리자):
 - `user.id = community_report.handled_by_user_id` (nullable)

신고 ↔ 대상 엔티티 (다형 관계, FK 비강제)

- 신고 대상은 게시글 또는 댓글이 될 수 있다.
- 대상은 `target_type + target_id` 조합으로 식별한다.

논리적 연결 규칙

- `target_type = 'post'`
 - `community_report.target_id = community_post.id`
- `target_type = 'comment'`
 - `community_report.target_id = community_comment.id`

본 관계는 DB 레벨 FK로 강제하지 않으며,
신고 생성 시점에 애플리케이션 로직으로 대상 존재 여부를 검증한다.

관리자 행위 로그

- `user.id = admin_action_log.actor_user_id`
→ 모든 운영 행위는 행위자(`user`)와 연결된다.

2.4 법령

- `laws.law_id = law_units.law_id`
→ 하나의 법령에 여러 조문 단위가 소속됨
- `law_units.doc_id = law_units.parent_id`
→ 조문 간 상·하위 구조(조 → 항 → 호 → 목) 표현

2.5 분쟁기준

- `criteria.source_id = criteria_units.source_id`
→ 하나의 분쟁기준 원천에 여러 단위 레코드가 소속됨

정규화 및 데이터 중복 방지 설계

본 설계는 **3차 정규형(3NF)**을 기준으로 데이터 중복을 최소화하였다.

3.1 사용자 관련 정규화

- `user` : 내부 사용자 식별, 상태(`status`), 권한(`role`) 관리
- `user_auth_provider` : 외부(소셜) 인증 정보 관리
- `user_session` : 로그인 유지(세션) 및 refresh token 관리

👉 인증/세션 책임을 분리하여 변경 영향 최소화

👉 `user_profile` 은 현재 DBML 범위에 포함되지 않음(커뮤니티 영역에서 별도 관리)

3.2 챗봇 데이터 정규화

- `chat` : 대화방 메타데이터(제목/상태/정렬용 `last_message_at`) + LangSmith thread 연결(`langsmith_thread_id`)
- `chat_message` : 메시지 본문(역할 `role` , 순서 `sequence`) + 생성 주체 추적(`agent_id`)

👉 메시지에 사용자/대화방 정보를 중복 저장하지 않고 FK로 참조 (`chat_message.chat_id` → `chat.id` , `chat.user_id` → `user.id`)

3.3 커뮤니티 데이터 정규화

(1) 게시글 / 댓글 분리

- 게시글(`community_post`)과 댓글(`community_comment`)을 별도 엔티티로 분리
- 게시글 단위 조회와 댓글 단위 관리 책임 분리

(2) 대댓글 자기 참조 구조

- `parent_comment_id`를 통한 자기 참조(Self FK) 구조 적용
- 댓글글 1단까지만 허용하여 계층 복잡도 제한

(3) 신고 데이터 분리

- 신고 정보를 `community_report` 테이블로 분리
- 게시물/댓글 도메인 로직과 운영 로직 분리

(4) 운영 로그 분리

- 관리자 행위 이력을 `admin_action_log` 테이블로 분리
- 도메인 데이터와 감사(Audit) 데이터 분리

3.4 중복 방지 제약 요약

- 소셜 계정 중복 방지: `(provider, provider_user_id)` UNIQUE
- 사용자-인증 1:1 강제: `user_auth_provider.user_id` 단일 참조
- 닉네임 중복 방지: `nickname_norm` UNIQUE
- 좋아요 중복 방지:
 - `(post_id, user_id)` UNIQUE
 - `(comment_id, user_id)` UNIQUE

3.5 법령 데이터 정규화

- `laws`
: 법령 단위의 **정적 메타데이터** 관리
(법령명, 소관 부처, 공포·시행일, 개정 유형 등)
 - `law_units`
: 법령을 구성하는 조·항·호·목 단위의 실제 조문 내용 관리
계층 구조 및 참조 관계 포함
- 👉 조문 단위 검색의 독립성 확보

3.6 분쟁기준 데이터 정규화

- `criteria`
: 분쟁기준 문서 단위의 **정적 메타데이터** 관리
(별표 구분, 지침 유형, 데이터 출처 식별)
- `criteria_units`
: 분쟁기준을 구성하는 **검색·활용 단위 레코드** 관리
(품목, 해결기준, 조건, 설명 등 실제 활용 텍스트 포함)
- 분쟁기준 원천과 단위 레코드를 분리함으로써
동일 문서 메타데이터의 반복 저장을 방지하고,
기준 내용 변경 시 영향을 최소화하였다.
- 각 단위 레코드는 `criteria_units.unit_id`를 통해 독립적으로 식별되며,
원천 정보는 `criteria.source_id` 참조로만 연결하여
내용 중심 조회 및 RAG 검색에 최적화된 구조를 유지한다.